BÁO CÁO KĨ THUẬT: PHÂN TÍCH VÀ THIẾT KẾ DEVICE DRIVER

ASSIGNMENT 01 - EMBEDDED LINUX SYSTEMS

THÔNG TIN DỰ ÁN

- **Môn học:** Hệ thống Linux Nhúng (Embedded Linux Systems)
- **Bài tập:** ASSIGNMENT-01: How to Write a Device Driver
- **Kiến trúc mục tiêu:** ARMv7-A – Cortex-A9 (QEMU vexpress-a9)
- **Phiên bản Linux Kernel:** 5.15 LTS
- **Cross Compiler:** arm-linux-gnueabihf-gcc (GCC 9+)
- **Thành viên:**

LỚP	IC2009
NHÓM	3
TÊN	MSSV
Trần Gia Thành	SE203763
Nguyễn Đăng Khoa	SE204014
Châu Nhật Nguyên	SE203853
Võ Hoàng Trí	SE204383

PHẦN I: MÔI TRƯỜNG PHÁT TRIỂN VÀ BỐ TRÍ MÃ NGUỒN

1. Môi trường Thực thi và Biên dịch Chéo

- **Host OS:** Ubuntu 22.04 LTS 64-bit.
- **Target Emulator:** QEMU ARM (`qemu-system-arm`) giả lập bo mạch `vexpress-a9` (Dual-core Cortex-A9, RAM 512MB).
- **Root Filesystem:** Built từ BusyBox 1.35.0 (Static linking) đóng gói dạng `initramfs.cpio.gz`.

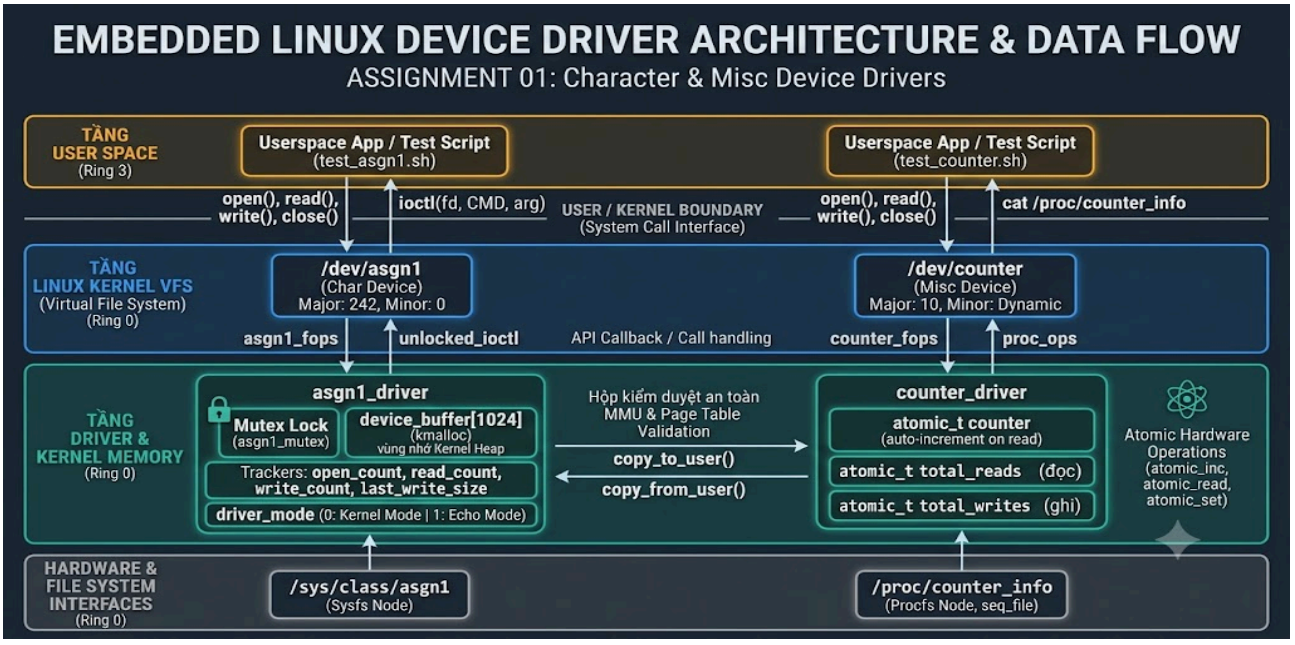
2. Cấu trúc Thư mục (Repository Structure)

```
Assignment01/
├── task1_ioctl/
│   ├── asgn1_driver.c       # Character Driver mở rộng hỗ trợ IOCTL & Mutex
│   ├── asgn1_ioctl.h        # Header định nghĩa Magic Number, Struct Stats & 4 lệnh IOCTL
│   ├── Makefile             # KBuild Makefile biên dịch chéo cho ARMv7
│   └── test_asgn1.sh        # Script kiểm thử tự động trên QEMU
├── task2_misc/
│   ├── counter_driver.c     # Misc Device Driver quản lý biến đếm atomic_t
│   ├── Makefile             # KBuild Makefile biên dịch counter_driver.ko
│   └── test_counter.sh      # Script kiểm thử /dev/counter & /proc/counter_info
├── logs/
│   ├── test_asgn1_output.txt          # Terminal Log chạy test Task 1
│   ├── test_counter_output.txT        # Terminal Log chạy test Task 2
│   └── dmesg_full.txt           # Kernel log (dmesg)
└── bao_cao/
    └── MSSV_Asgn01_BaoCao.pdf
```

PHẦN II: PHÂN TÍCH THIẾT KẾ VÀ KIẾN TRÚC DRIVER (TASK 3)

1. Sơ đồ Kiến trúc Tổng thể và Luồng Dữ liệu

Sơ đồ thể hiện mối liên kết giữa Userspace App, System Call Layer, VFS, `file_operations` callback, Mutex Protection, Bộ đệm nội bộ Kernel, Biến đếm Atomic, cùng giao diện procfs (`/proc`) và sysfs (`/sys`):



Luồng xử lý chi tiết:

- **Write/Read Path (`asgn1_driver`):** Khi Userspace gọi `write()`, VFS chuyển giao tới `asgn1_write()`. Driver khóa `mutex_lock(&asgn1_mutex)` để ngăn chặn race condition, cấp phát bộ đệm tạm qua `kmalloc`, nạp dữ liệu bằng `copy_from_user()`, căn cứ theo `driver_mode` (0: Kernel mode với tiền tố `[KERNEL]`, 1: Echo mode) để định dạng bộ đệm nội bộ, sau đó mở khóa Mutex.
- **IOCTL Control Path (`asgn1_driver`):** Khi ứng dụng gọi `ioctl(fd, CMD, arg)`, lệnh chuyển qua `unlocked_ioctl`. Driver phân loại theo switch-case đối với 4 lệnh:
    a. `ASGN1_RESET_BUFFER`: Xóa bộ đệm và reset toàn bộ biến đếm về 0.
    b. `ASGN1_GET_STATS`: Đóng gói `struct asgn1_stats` và truyền về người dùng bằng `copy_to_user()`.
    c. `ASGN1_SET_MODE`: Nhận giá trị mode từ người dùng bằng `copy_from_user()`.
    d. `ASGN1_GET_VERSION`: Truyền chuỗi định danh phiên bản driver về người dùng.
- **Counter Path (`counter_driver`):** Đọc `/dev/counter` kích hoạt `counter_read()`, ghi nhận giá trị hiện tại, tự động tăng `counter` và `total_reads` bằng các hàm nguyên tố `atomic_inc()`, sau đó chuyển chuỗi ASCII về Userspace. Đọc `/proc/counter_info` sử dụng `seq_file` để xuất thông số mà không gây lock hệ thống.

2. So sánh 3 Loại Device Driver Trong Linux Kernel

Tiêu chí So sánh	Character Device Driver	Block Device Driver
Giao diện VFS	Dùng `struct file_operations` qua file node `/dev`	Dùng `struct block_device_operations`
Đơn vị dữ liệu	Luồng ký tự nối tiếp (`Byte Stream`)	Khối dữ liệu định dạng sẵn (`Block / Sector`: 512B, 4096B)
Cơ chế Buffering	Trực tiếp giữa User & Kernel Buffer	Bắt buộc sử dụng Buffer Cache / Page Cache
Phương thức truy cập	Tuần tự (Sequential Access)	Ngẫu nhiên (Random Access theo chỉ số Sector)
Ví dụ thực tế	UART Console, Cảm biến I2C/SPI, Virtual Devices	Thẻ nhớ SD/eMMC, SATA SSD, NVMe, USB Flash

Lý do lựa chọn thiết kế:

- **Character Driver (`asgn1_driver`):** Thích hợp cho các thiết bị truyền nhận dữ liệu dòng ký tự không cố định kích thước, đòi hỏi độ trễ thấp và phản hồi tức thì.
- **Misc Driver (`counter_driver`):** Giúp tối ưu tài nguyên Kernel bằng cách dùng chung Major Number 10 (`MISC_MAJOR`) thay vì chiếm dụng một Major Number riêng biệt trong bảng đăng ký thiết bị.

3. Phân tích Các Quyết định Thiết kế

3.1. `register_chrdev` vs `misc_register`

- `register_chrdev()` cấp phát một Major Number riêng cho driver và quản lý tới 256 Minor Numbers. Phù hợp với thiết bị nhiều kênh (như đa cổng UART), nhưng yêu cầu lập trình viên tự đăng ký `class_create` và `device_create` trong sysfs.
- `misc_register()` đơn giản hóa việc đăng ký bằng cách xếp thiết bị vào nhóm Miscellaneous Devices (Major 10). Kernel tự động tạo device node trong `/dev` và cấp Minor động (`MISC_DYNAMIC_MINOR`), giúp tiết kiệm mã nguồn và Major Number.

3.2. Tại sao sử dụng `atomic_t` cho Counter Driver?

Trên hệ thống đa nhân (SMP) hoặc đa tiến trình, thao tác `counter++` không phải là lệnh nguyên tố mà gồm 3 bước Assembly: Read RAM → Modify Register → Write RAM. Nếu 2 tiến trình truy cập đồng thời, race condition sẽ làm sai lệch dữ liệu.

Sử dụng atomic_t cùng các hàm nguyên tố (atomic_inc, atomic_read, atomic_set) đảm bảo thao tác được khóa ở cấp độ lệnh phần cứng mà không phải chịu chi phí hao tổn hiệu năng (overhead) như khóa Mutex hay Spinlock.

3.3. Cơ chế IOCTL Magic Number và Macros

Mã lệnh IOCTL 32-bit gồm 4 vùng thông tin:

- **Magic Number (Type - 8 bits):** Định danh duy nhất cho driver (`ASGN1_MAGIC 'a'`).
- **Sequence Number (NR - 8 bits):** Số thứ tự lệnh (1, 2, 3, 4).
- **Direction (Dir - 2 bits):** Hướng truyền dữ liệu (`_IOC_NONE`, `_IOC_READ`, `_IOC_WRITE`).
- **Size (13-14 bits):** Kích thước của struct/kiểu dữ liệu tham số.

Macro ứng dụng:

- `_IO(ASGN1_MAGIC, 1)`: Lệnh điều khiển không kèm dữ liệu (`ASGN1_RESET_BUFFER`).

- `_IOR(ASGN1_MAGIC, 2, struct asgn1_stats)`: Lệnh đọc thông số từ Kernel về Userspace (`ASGN1_GET_STATS`).
- `_IOW(ASGN1_MAGIC, 3, int)`: Lệnh ghi cấu hình từ Userspace vào Kernel (`ASGN1_SET_MODE`).
- `_IOR(ASGN1_MAGIC, 4, char[64])`: Lệnh đọc chuỗi phiên bản (`ASGN1_GET_VERSION`).

4. Kết quả Kiểm thử và Nhật ký Khắc phục Lỗi

4.1. Kết quả kiểm thử Task 1 (test_asgn1.sh)

- Đã nạp thành công `asgn1_driver.ko` (Major 242).
- Thực thi ghi/đọc dữ liệu ở Echo Mode (Mode 1) trả về đúng chuỗi đầu vào.
- Thực thi đổi Mode sang Kernel Mode (Mode 0) bổ sung chính xác tiền tố `[KERNEL]`.
- Lệnh `ASGN1_GET_STATS` trả về đúng các thông số `open_count`, `write_count`, `read_count`, `buffer_len`.
- Lệnh `ASGN1_RESET_BUFFER` xóa sạch bộ đệm và reset bộ đếm về 0.

4.2. Kết quả kiểm thử Task 2 (test_counter.sh)

- Đã nạp thành công `counter_driver.ko` tạo node `/dev/counter`.
- Đọc `/dev/counter` tự động tăng giá trị đếm lên 1 sau mỗi lần đọc.
- Ghi giá trị `'100'` vào `/dev/counter` cập nhật chính xác biến đếm.
- Đọc `/proc/counter_info` hiển thị chi tiết Current Counter, Total Reads, Total Writes.

4.3. Nhật ký Khắc phục Lỗi Kỹ thuật

- **Sự cố 1 (Stack Frame Warning):** Compiler báo warning `frame size of 1032 bytes is larger than 1024 bytes` do khai báo mảng cục bộ `char temp_buf[1024]` trên Stack.
    - *Khắc phục:* Chuyển sang cấp phát bộ đệm trên Heap bằng `kmalloc(len + 1, GFP_KERNEL)` và giải phóng bằng `kfree()`.
- **Sự cố 2 (Assembler Unsupported Instruction):** Trình biên dịch báo `selected processor does not support isb in ARM mode`.
    - *Khắc phục:* Bổ sung tham số kiến trúc mục tiêu ARMv7-A vào Makefile: `EXTRA_CFLAGS += -march=armv7-a -mfpu=vfpv3-d16`.

PHẦN III: LỜI GIẢI CÂU HỎI LÝ THUYẾT (TASK 4)

Câu 1: Reference Counting & Cơ chế Unload Module

- **Cơ chế:** Kernel tự động tăng reference count khi tiến trình mở file thiết bị qua `open()` (`try_module_get()`) và giảm reference count khi đóng file qua `release()` (`module_put()`).
- **Sự cố rmmod:** Nếu reference count > 0, lệnh `rmmod` sẽ bị từ chối với lỗi `Module asgn1_driver is in use`.
- **Cưỡng chế Unload & Rủi ro:** Có thể dùng `rmmod -f` (nếu kernel cấu hình `CONFIG_MODULE_FORCE_UNLOAD=y`). Tuy nhiên, việc gỡ module khi process đang giữ open sẽ làm giải phóng vùng nhớ mã nguồn driver. Khi process thực hiện `read()`/`write()` tiếp theo, CPU sẽ truy cập vùng nhớ rác gây ra **Kernel Page Fault** và kéo theo **Kernel Panic/Crash** toàn hệ thống.

Câu 2: `copy_to_user` / `copy_from_user` vs `memcpy`

- **Lý do MMU:** Kernel Space và User Space có hai không gian địa chỉ ảo độc lập. Con trỏ từ Userspace có thể trỏ tới vùng nhớ chưa được map trang (Unmapped Page) hoặc đã bị Paged-out ra Swap disk.
- **Tác hại của memcpy:** Nếu dùng `memcpy()` trực tiếp, khi gặp địa chỉ không hợp lệ, MMU sẽ phát sinh lỗi Page Fault trong Kernel Mode. Vì không có cơ chế bắt lỗi, Kernel sẽ sụp đổ ngay lập tức.
- **Ưu điểm của `copy_to_user`:** Tích hợp kiểm tra `access_ok()` và có Bảng xử lý ngoại lệ (Exception Table Fixup). Nếu xảy ra Page Fault, Kernel bắt lỗi an toàn và trả về mã lỗi `-EFAULT` về Userspace mà hệ thống vẫn vận hành bình thường.

Câu 3: MTD Subsystem vs Block Layer

- **Đặc tính Phần cứng:** Block Devices (SD card, SSD) tích hợp controller phần cứng (FTL) che giấu bản chất Flash và cho phép ghi đè sector. Trong khi đó, Raw Flash (NAND/NOR thô) nối trực tiếp bus vi xử lý, không có FTL.
- **Lý do dùng MTD Subsystem:**
    a. *Erase-before-write:* Raw Flash đòi hỏi phải xóa toàn bộ Erase Block (128KB-512KB) trước khi ghi đè dữ liệu mới.
    b. *Bad Block Management:* Theo dõi và đánh dấu các khối hỏng trên chip Flash thô.
    c. *Wear Leveling:* MTD kết hợp với JFFS2/UBIFS giúp phân bổ đều thao tác ghi/xóa, bảo vệ tuổi thọ ô nhớ Flash.

Câu 4: Cập nhật Con trỏ *ppos Trong Hàm Read

- **Cơ chế VFS:** Khi người dùng thực thi `cat /dev/asgn1`, VFS gọi hàm `read()` trong một vòng lặp liên tục cho đến khi hàm trả về `0` (tín hiệu EOF - End of File).
- **Hậu quả nếu không cập nhật *ppos:** Nếu driver trả về số byte đọc được nhưng không cộng dồn giá trị vào `*ppos` (`*offset += bytes_read`), ở lần gọi tiếp theo, VFS vẫn truyền `*offset = 0`. Driver lại tiếp tục đọc dữ liệu từ đầu và trả về số byte dương. Kết quả là VFS **không bao giờ nhận được EOF (0)**, khiến chương trình ở Userspace bị treo trong vòng lặp vô hạn (Infinite Loop).